

Motion Optimization of Wheeled Soccer Robot Using Kalman Filter on YOLOv8-Based Object Tracking

Motion Optimization of Wheeled Soccer Robot Using Kalman Filter on YOLOv8-Based Object Tracking

ABSTRACT

The Indonesian Wheeled Soccer Robot Contest (KRSBI-Beroda) demands highly responsive, real-time ball detection and tracking within dynamic and visually complex environments. However, many participating university teams face severe hardware limitations, relying primarily on consumer-grade laptops for vision processing. While deep learning models like You Only Look Once version 8 (YOLOv8) offer exceptional detection accuracy, their deployment as continuous, single-frame detectors on such resource-constrained hardware leads to severe computational bottlenecks. This limitation directly results in low frame rates (FPS), mechanical command jitter, and critical tracking failures when the ball undergoes moderate or momentary occlusion. To overcome these challenges without requiring expensive hardware upgrades, the main contribution of this work is the development of a lightweight, hybrid tracking architecture. This system integrates YOLOv8 with a Kalman Filter and an adaptive frame-skipping mechanism. In this proposed architecture, YOLOv8 serves as an intermittent feature detector, while the Kalman Filter functions as a continuous predictive spatial memory, estimating the ball's trajectory during the skipped frames. Experiments were rigorously conducted on a robot platform equipped with an omnidirectional camera and an ASUS K401U laptop. The optimized system effectively boosts the average processing speed from 13.60 to 38.77 FPS (a 185% increase), comfortably fulfilling real-time requirements. Furthermore, the predictive capability of the Kalman Filter eliminates tracking loss during moderate and momentary occlusions (0 occurrences). Mechanically, the integration reduces motion command jitter by 69% and significantly cuts the overall goal-scoring mission completion time by 62% (from 15.17 s to 5.76 s). These results demonstrate that the proposed software-based optimization offers a highly cost-effective solution for KRSBI teams to achieve stable and responsive ball tracking, successfully bridging the gap between heavy deep learning detection and lightweight recursive state estimation in dynamic robotic environments.

Keywords : KRSBI-Beroda, YOLOv8, Kalman Filter, Object Tracking, Frame Skipping, Soccer Robot, Computer Vision.

Submitted:

*Article Info :
Reviewed:*

Accepted :

Copyright © 2022 – CSRID Journal. All rights reserved.

1. INTRODUCTION

The advancement of computer vision and deep learning technologies has driven rapid progress in autonomous robotics, particularly in the Indonesian Robot Contest (KRI), specifically the Indonesian Wheeled Soccer Robot Contest (KRSBI-Beroda) category [1]. In this competition, robots are required to detect and track ball movements in real-time to perform dribbling maneuvers and score goals. Failure in ball tracking, whether due to computational limitations or occlusion disturbances, can be fatal to the robot's performance on the field [2].

In the early stages of KRSBI-Beroda development, many teams relied on the HSV color filtering method for ball detection. This method is computationally lightweight but highly sensitive to changes in light intensity, making it unreliable in dynamic field conditions [2]. This weakness becomes critical because KRSBI-Beroda matches often take place under varying and uncontrolled lighting conditions [1].

With technological advancements, Convolutional Neural Network (CNN)-based methods such as You Only Look Once (YOLO) have been widely adopted by KRSBI teams to improve detection accuracy [3, 4]. YOLO can detect objects with high precision even under varying lighting conditions [5]. The YOLOv8 architecture used in this study is the latest version with improvements in model efficiency, feature extraction, and computational speed [5, 6]. Research by Farhan & Candra [3] demonstrated that the YOLOv8 model achieves an accuracy of 95.87% for ball and goal detection on KRSBI-Beroda robots.



Figure 1. Illustration of KRSBI-Beroda Match Environment
(Source: kontesrobotindonesia.id)

However, implementing YOLOv8 on limited hardware, a common constraint among university robotics teams using consumer-grade laptops, introduces new problems: high computational load causes low frame rate (FPS) and lag [3]. As a result, the robot often loses track of the ball when it moves quickly or when the ball is partially occluded [8]. Occlusion is a major cause of increased false-negative detection rates, which can degrade overall tracking performance [9]. This issue is universal across KRSBI participants who face similar budget and hardware constraints.

To overcome these limitations, researchers have combined deep detectors with recursive Bayesian filters. Saputra [10] integrated SIFT features with a Kalman Filter for ball tracking on KRSBI robots, achieving mean absolute errors of 1.06 px (x) and 7.34 px (y), but SIFT is less robust than learned deep features. Sholehurrohman et al. [11] compared Kalman, Particle, and Correlation filters, finding Kalman Filter gave the highest accuracy (96.89%), but their work did not address real-time hardware limitations. Taylor et al. [12] showed that combining Kalman Filter with a cropped search window reduces processing time significantly, yet their system relied on color segmentation rather than a state-of-the-art detector. Lee [13] proposed confidence-guided frame skipping to speed up heavy trackers by 1.5 $\times$, but the frame skipping was not integrated with a motion model, risking drift during fast motion. In contrast, our work proposes a tightly coupled architecture where YOLOv8 provides high-quality measurements, and a Kalman Filter with a constant-velocity model predicts the ball's state during skipped frames, bridging the gap between deep detection and lightweight temporal prediction.

The main contribution of this paper is a novel hybrid tracking system that:

1. Integrates YOLOv8 with a Kalman Filter via an adaptive frame-skipping schedule (SL = 3) to maintain real-time throughput on low-cost hardware.

2. Implements a loss-tolerance mechanism that keeps predicting the ball trajectory for up to 5 seconds during occlusion, achieving zero tracking loss.
3. Demonstrates through extensive experiments quantitative improvements in FPS, occlusion resilience, motion jitter reduction, and mission completion time.

The proposed method offers a practical software-based solution that can be readily adopted by other KRSBI teams or similar small-robot competitions facing similar resource limitations. This study proposes a tightly coupled hybrid tracking architecture that integrates intermittent deep learning detection with continuous state estimation, enabling real-time performance on resource-constrained hardware without sacrificing tracking robustness.

2. METHOD

A. Research Design and Model Preparation

This study employs a quantitative experimental approach by comparing two systems: a Baseline (pure YOLOv8, inference every frame) and an Optimized system (YOLOv8 + Kalman Filter with frame skipping). The robot platform consists of an ASUS K401U laptop (Intel Core i3, integrated GPU, 4 GB RAM) running ROS, an Arduino Mega 2560 for motor control, and an omnidirectional camera capturing 360° views. The YOLOv8 model was pre-trained on a custom dataset of 1,125 images taken from the robot's own omnidirectional camera, covering various ball positions, lighting, and backgrounds, achieving 95.87% accuracy [3].

Prior to integration, the YOLOv8 model was pre-trained using a specialized dataset captured directly from the robot's omnidirectional camera. This ensures the model's resilience against the unique radial distortion and varying lighting conditions of the field. The output of this detection model consists of bounding box parameters (x_c, y_c, w, h) along with a confidence score. In the optimized architecture, these exact parameters serve directly as the noisy measurement vector (Z_k) input for the Kalman Filter's correction stage.

B. Optimized System Architecture and Frame Skipping

In the optimized system, the tracking architecture is designed to reduce computational load without sacrificing tracking continuity. The system applies frame skipping with an interval of $SL = 3$, meaning YOLOv8 inference is performed on the 1st frame, while the 2nd and 3rd frames rely entirely on predictions from the Kalman Filter.

The state vector X is defined as:

$$X = [x, y, v_x, v_y, w, h]^T \quad (1)$$

where (x, y) is the center coordinate of the ball, (v_x, v_y) is the object's velocity, and (w, h) are the dimensions of the bounding box. The motion model used is constant velocity with the transition matrix F :

$$F = \begin{bmatrix} 1 & 0 & dt & 0 & 0 & 0 \\ 0 & 1 & 0 & dt & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

The observation matrix H only reads (x, y, w, h) because YOLO does not directly provide velocity data:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

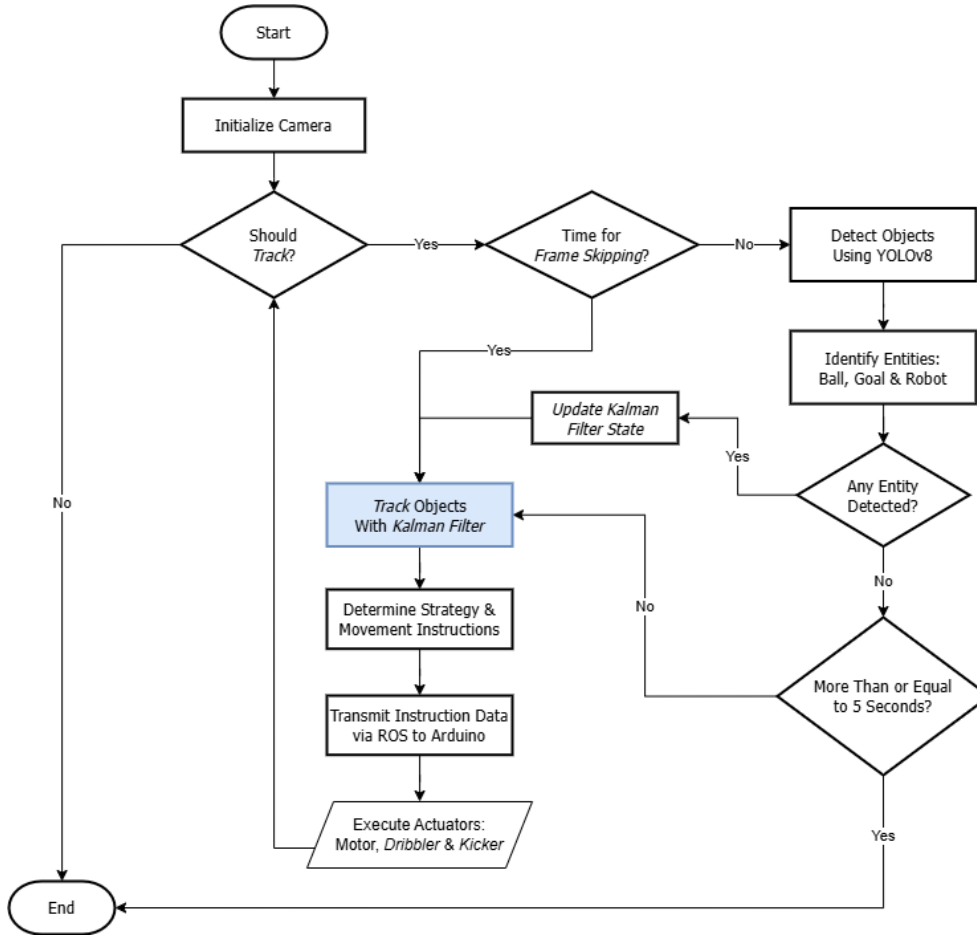


Figure 2. Workflow of the Optimized Tracking System with Frame Skipping and Kalman Filter

The workflow illustrated in Figure 2 demonstrates the adaptive tracking mechanism. To handle object occlusion, the system incorporates a loss-tolerance threshold. If YOLO fails to detect the ball on a scheduled frame (due to visual obstruction), the Kalman Filter bypasses the correction phase and purely executes the prediction phase using its last known velocity vector. The track is only discarded if the object remains undetected for a continuous duration of 5 seconds, providing robust continuity during momentary occlusions.

C. Software Implementation

The system is implemented using:

- **ROS (Robot Operating System)** [14]: For inter-node communication and robot system management.
- **OpenCV v3.4** [15]: For image processing and visualization
- **NumPy v2.3** [16]: For Kalman Filter matrix computation.
- **Ultralytics YOLOv8** [7]: For object detection inference.

D. Vision-Based Motion Control and Grid Localization

A critical component of the methodology is translating the continuous pixel coordinates (x, y) generated by the tracking system into discrete mechanical commands for the robot's actuators. Because the robot utilizes an omnidirectional camera that captures a 360° distorted field of view, the system implements a Grid Localization algorithm to map the spatial environment accurately. In this approach, the visual frame is logically divided into an (n × m) grid matrix through an initial calibration process.

Once the Kalman Filter outputs the predicted coordinates of the ball, the algorithm maps these continuous coordinates to a specific grid zone. Each zone is pre-assigned a corresponding navigational instruction. For instance, if the ball is located in the top-center zone of the frame, the system issues a "forward" command to approach the target. If the ball falls within the left or right zones, it issues "turnLeft" or "turnRight" commands to adjust the robot's heading. Furthermore, if no object is tracked within the frame, a "search" rotation is automatically initiated. These string-based commands are continuously published via the *rosserial* communication protocol. The laptop, acting as the high-level processor, transmits these messages to the Arduino Mega 2560 microcontroller, which subsequently translates the strings into precise Pulse Width Modulation (PWM) signals to drive the omnidirectional wheels.

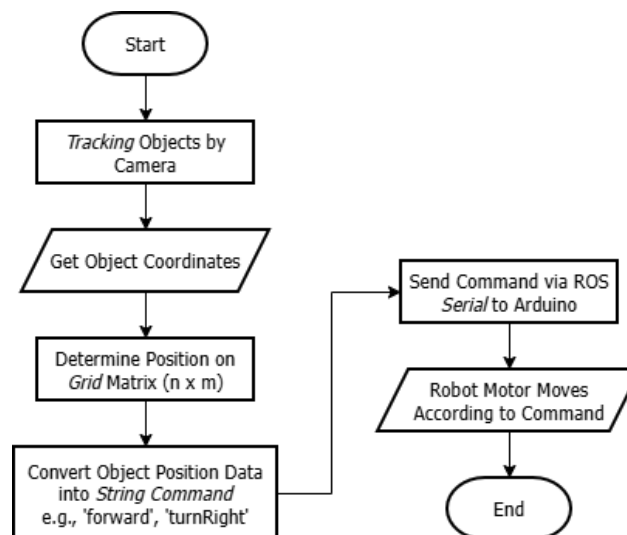


Figure 3. Flowchart of the Grid Localization Algorithm for Motion Control

E. Evaluation Scenarios

Experiments were conducted in a laboratory with a green carpet and static obstacles to mimic match conditions. Key testing parameters:

- Camera resolution: 640×480 pixels, 30 fps capture.
- Kalman Filter time step $dt = 1/\text{current FPS}$.
- Static test: 30 seconds recording, 10 repetitions, ball stationary at center (2 m distance).
- Occlusion test: cardboard pieces covering 30% or 60% of ball area; momentary trial used a physical obstacle blocking the ball for 1–2 seconds while it moved at approx. 0.5 m/s.
- Dynamic test: robot started 2 m from ball, had to dribble and shoot; 5 repetitions per system.
- Metrics: FPS (frames per second), Tracking Loss Count (number of tracking ID terminations), Total Loss Duration (seconds without any bounding box), motion jitter (frequency of direction-change commands), mission completion time, and goal rate.



Figure 4. The laboratory environment utilized for system evaluation, featuring a simulated match field and physical obstacles to test the robot's vision capabilities

3. RESULT AND DISCUSSION

A. Computational Performance Analysis



Figure 5. Visual outcome of the static test on the baseline system showing a low frame rate (avg. 13.60 FPS)

Figure 5 and Figure 6 provide a direct visual comparison of the robot's vision interface during the static test. As depicted in Figure 5, forcing the YOLOv8 model to perform full deep learning inference on every single frame creates a severe computational bottleneck on the ASUS K401U laptop, limiting the processing speed to an average of only 13.60 FPS (detailed in Table 1).



Figure 6. Visual outcome of the static test showing stable detection and enhanced FPS (avg. 38.77) in the optimized system

As depicted in Figure 6, the graphical user interface of the robot's vision system demonstrates the real-time tracking capabilities of the optimized system. During the static test, the system consistently maintained a high frame rate, averaging around 38.77 FPS on the interface. This visual evidence highlights the efficiency of the frame-skipping strategy, which successfully prevents computational bottlenecks without losing track of the stationary ball.

Based on the static test results summarized in Table 1, the optimized system successfully increased the average FPS significantly.

Table 1. Comparison of Average FPS in Static Test

Repetition	Average FPS	
	Baseline System	Optimized System
1	13.91	41.28
2	14.34	36.29
3	13.54	37.61
4	13.73	43.16
5	13.52	34.38

Repetition	Average FPS	
	Baseline System	Optimized System
6	13.48	35.30
7	12.68	40.05
8	12.52	38.83
9	15.09	44.00
10	13.21	36.81
Average	13.60	38.77

The 185% increase in FPS occurs because the Kalman Filter is significantly more computationally lightweight ($O(n^2)$ simple matrix operations) compared to the YOLOv8 neural network inference, which intensively consumes CPU/GPU resources. With an average FPS of 38.77, the system meets the real-time requirement (>30 FPS), and the robot can react more quickly to dynamic ball movements. This result is consistent with the research by Taylor et al. [12], which proved that Kalman Filter integration significantly reduces processing time.

B. Occlusion Resilience

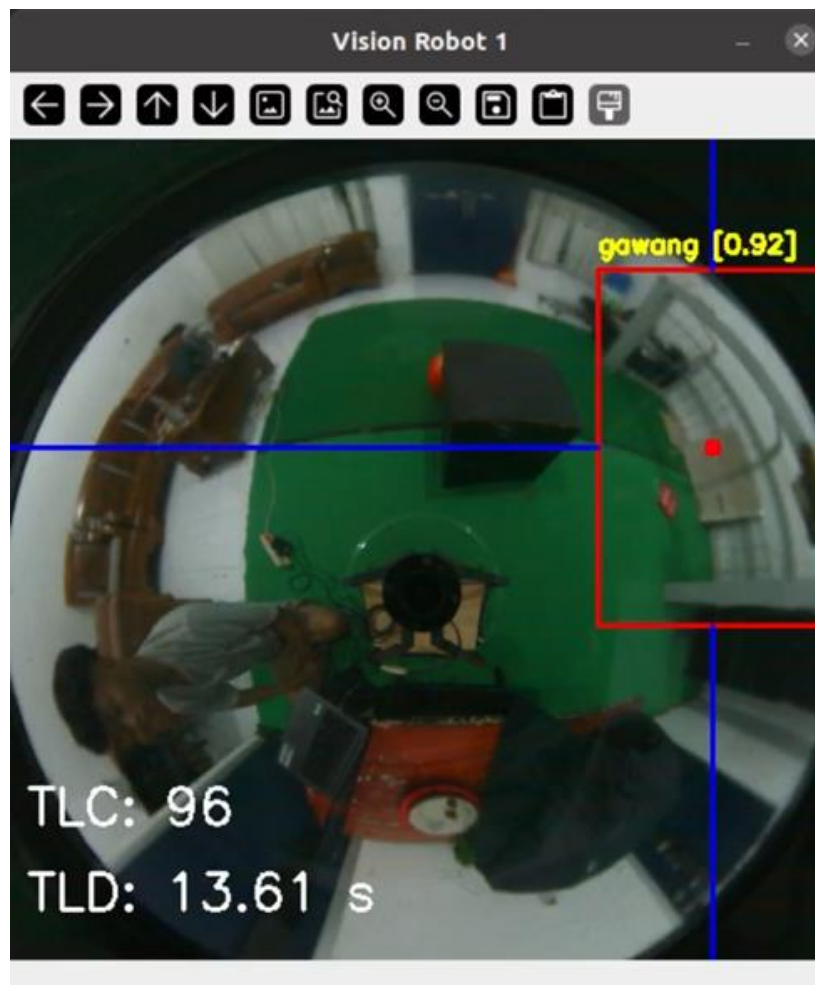


Figure 7. Baseline system failing to detect ball under 60% occlusion (TLC: 96)

Figure 7 and Figure 8 illustrate the direct comparison between the two systems under a moderate 60% occlusion scenario. As observed in Figure 7, the baseline system completely fails to draw a bounding box around the partially hidden ball, accumulating a Tracking Loss Count (TLC) of 96. Conversely, Figure 8 demonstrates the optimized system's robustness; it successfully retains the bounding box (indicated by the [kf] label) by relying on the Kalman Filter's ongoing estimation, keeping the TLC at a perfect 0.



Figure 8. Optimized system maintaining tracking under 60% occlusion (TLC: 0)

To further challenge the system's predictive capabilities, a momentary occlusion scenario was introduced, as illustrated in Figure 9, where the ball rolls behind a solid obstacle for 1 to 2 seconds. The visual logs reveal a stark contrast. The baseline system (Figure 10) suffers a Total Loss Duration (TLD) of 2.89 seconds, leaving the robot effectively blind. In sharp contrast, the optimized system (Figure 11) maintains an uninterrupted trajectory projection, yielding a TLD of 0.00 seconds. This proves that the Kalman Filter's velocity parameters (v_x , v_y) effectively bridge the visual gap.



Figure 9. Illustration of the Momentary Occlusion scenario where the ball moves behind an obstacle

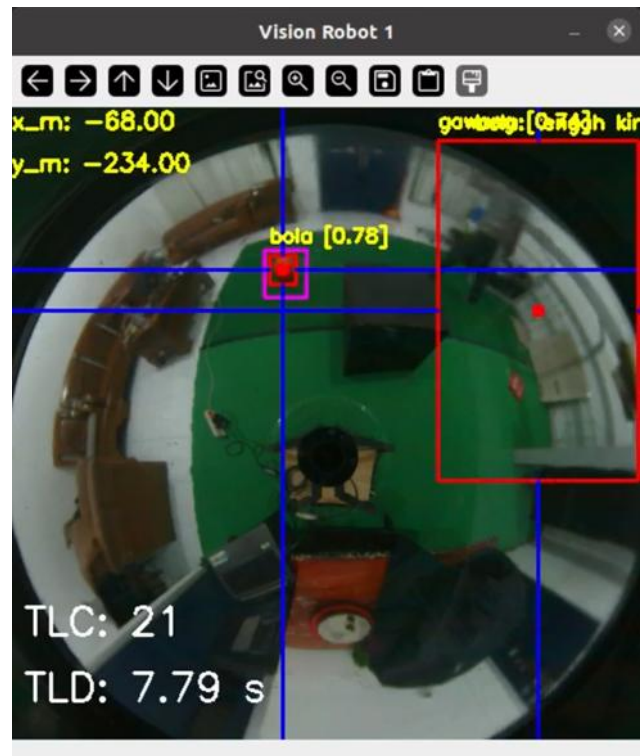


Figure 10. Baseline system losing track during momentary occlusion (TLD: 2.89 s)

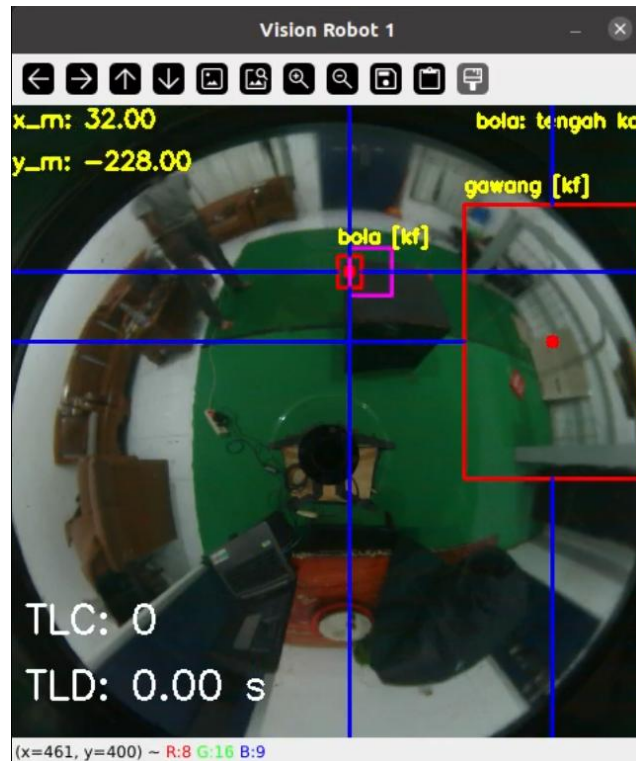


Figure 11. Optimized system successfully predicting ball trajectory (TLD: 0.00 s)

In Table 2, the baseline system significantly lost track of the ball during moderate and momentary occlusion. In contrast, the optimized system recorded 0 (zero) tracking loss in all scenarios.

Table 2. Comparison of Tracking Loss in Occlusion Test

Scenario	Baseline System		Optimized System	
	Loss Count	Duration (s)	Loss Count	Duration (s)
Mild Occlusion (30%)	0	0	0	0
Moderate Occlusion (60%)	96	13.94	0	0
Momentary Occlusion (1-2 s)	6	2.89	0	0

These results prove that the Kalman Filter functions as "spatial memory" for the robot. When the ball is occluded and YOLO fails to detect it, the Kalman Filter continues to project the ball's trajectory based on velocity estimation (v_x, v_y) from the previous state vector [10]. This capability is crucial for avoiding navigational confusion when the robot collides with opponents or the ball is momentarily obstructed. This finding aligns with the research by Takura et al. [17], which showed that the Kalman Filter effectively tracks occluded objects in video.

C. Motion Stability and Mission Time

The robot motion command log data presented in Table 3 shows a significant reduction in jitter.

Table 3. Comparison of Motion Command Frequency (5 Repetitions)

Command Type	Baseline System	Optimized System
turnLeft / turnLeftSlow	44 times	11 times

Command Type	Baseline System	Optimized System
turnRight / turnRightSlow	30 times	12 times
forward / forwardSlow	48 times	10 times
Total Direction Correction Commands	74 times	23 times

This 69% reduction in jitter directly impacts the efficiency of mission completion time, as shown in Table 4.

Table 4. Average Mission Completion Time in Dynamic Test

Repetition	Baseline System (seconds)	Optimized System (seconds)
1	12.400	7.254
2	19.277	5.350
3	12.986	5.987
4	19.139	5.615
5	12.057	4.638
Average	15.171	5.768

The optimized system consistently achieved a faster mission completion time, reducing the average duration by 62% (from 15.171 seconds to 5.768 seconds). This significant improvement highlights the system's ability to maintain a stable and continuous trajectory without the sudden stops or interruptions typically observed in the baseline system. Robot movement became smoother and more linear because the Kalman Filter estimation eliminates coordinate fluctuations that typically arise from YOLO detection noise or momentary lighting changes.

In terms of mission success, the goal rate metric was evaluated to assess the reliability of both systems. The experimental data shows that both the baseline and optimized systems achieved a 100% goal rate, successfully scoring 5 goals across 5 test repetitions. This indicates that in a controlled laboratory environment, the robot could eventually complete its task using either method. However, the qualitative difference lies in the execution; while the baseline system often required multiple corrective maneuvers due to detection jitter, the optimized system approached and kicked the ball with a much more linear and stable trajectory, resulting in a more efficient performance.

D. Comparative Discussion

To provide a comprehensive evaluation, this study compares the baseline and optimized systems across four key dimensions: computational efficiency, tracking robustness, motion stability, and task-level performance. Rather than exhibiting isolated improvements, the proposed method demonstrates consistent gains across all metrics, indicating a system-level optimization.

From a computational perspective, the optimized system achieves a substantial increase in processing speed, improving the average frame rate from 13.60 FPS to 38.77 FPS. This improvement is primarily attributed to the frame-skipping mechanism, which reduces the frequency of computationally expensive YOLOv8 inference while maintaining sufficient measurement updates. Unlike conventional approaches that rely on continuous deep learning inference, the proposed architecture strategically balances detection and prediction [10], enabling real-time performance on resource-constrained hardware without requiring model simplification or hardware upgrades.

In terms of tracking robustness, the contrast between the two systems is particularly evident under occlusion scenarios. The baseline system suffers significant tracking failures, especially under moderate (60%) and momentary occlusions, resulting in both high tracking loss counts and prolonged loss

duration. In contrast, the optimized system achieves zero tracking loss across all scenarios. This result highlights the role of the Kalman Filter as a predictive spatial memory, allowing the system to maintain continuous state estimation even when visual observations are temporarily unavailable. Compared to frame-skipping-only strategies reported in prior studies [13], which risk drift during skipped frames, the integration of a motion model in this work ensures trajectory continuity and prevents abrupt tracking discontinuities.

Beyond perception, the proposed method also significantly improves motion stability. The baseline system exhibits frequent direction changes due to noisy and discontinuous detections, leading to unstable robot behavior. By contrast, the optimized system reduces motion command corrections by 69%, demonstrating that the smoothed state estimation provided by the Kalman Filter effectively suppresses high-frequency fluctuations in the detected object position. This stabilization directly enhances the quality of control commands, resulting in smoother and more predictable robot motion.

At the task level, these improvements translate into a substantial reduction in mission completion time, decreasing from 15.17 seconds to 5.76 seconds on average. This 62% improvement reflects the cumulative effect of increased processing speed, uninterrupted tracking, and stable motion control. While both systems achieve a 100% goal rate in controlled conditions, the optimized system demonstrates significantly higher efficiency and consistency, indicating better suitability for dynamic and time-critical scenarios such as robotic soccer matches.

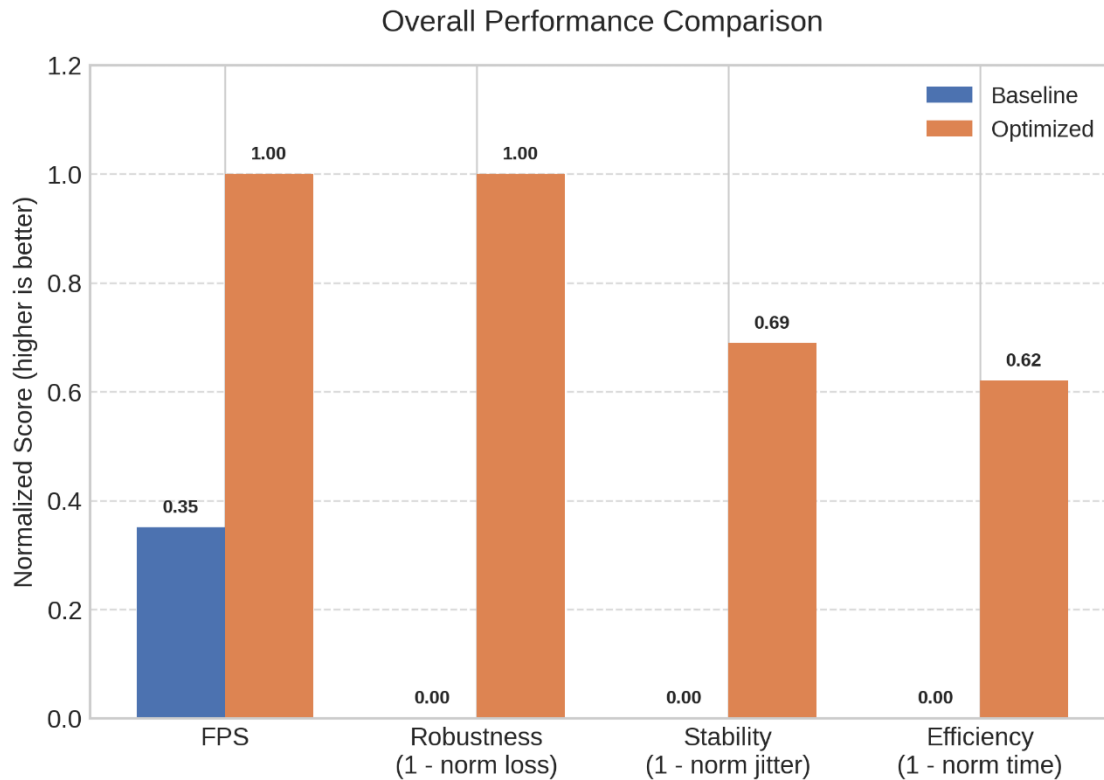


Figure 12. Normalized multi-metric comparison between the baseline and optimized systems across key performance dimensions, including computational efficiency (FPS), tracking robustness, motion stability, and task efficiency

To further consolidate the comparative analysis, a multi-metric visualization is presented in Figure 12, where all key performance indicators are normalized onto a common scale. This representation

enables a direct comparison across heterogeneous metrics, including frame rate, tracking robustness, motion stability, and task efficiency. As shown in the figure, the optimized system consistently achieves near-maximum normalized scores across all dimensions, whereas the baseline system exhibits significantly lower and uneven performance. Notably, the baseline system records near-zero performance in robustness due to substantial tracking loss under occlusion, while also underperforming in computational and control-related metrics. This visualization clearly highlights that the improvements introduced by the proposed method are not confined to a single aspect, but instead extend uniformly across the entire system pipeline. The balanced performance profile of the optimized system further reinforces the claim that the integration of intermittent detection and continuous prediction effectively eliminates the typical trade-offs observed in conventional approaches.

Importantly, the results show that the proposed method does not introduce trade-offs between performance metrics. In many existing approaches, improvements in computational efficiency often come at the expense of tracking accuracy or robustness [11, 18, 19]. However, the tightly coupled integration of intermittent detection and continuous prediction in this work enables simultaneous improvements across all evaluated dimensions. This distinguishes the proposed system from prior methods that address these challenges in isolation.

Overall, the findings confirm that the combination of YOLOv8, Kalman Filter-based state estimation, and frame skipping forms an effective hybrid tracking framework. By bridging the gap between computationally intensive deep learning detection and lightweight predictive modeling, the proposed approach provides a practical and scalable solution for real-time robotic vision systems operating under hardware constraints.

4. CONCLUSION

Based on the research results, the integration of the Kalman Filter into the YOLOv8 tracking system has proven effective in optimizing the motion of KRSBI-Beroda robots. The specific conclusions obtained are:

1. The optimized system successfully increased the average FPS by 185% (from 13.6 to 38.77), overcoming computational bottlenecks typical of consumer-grade hardware used in the competition.
2. The Kalman Filter provided perfect resilience against occlusion (0% tracking loss), ensuring the robot does not lose direction when the ball is obstructed—a critical advantage in dynamic gameplay.
3. Motion stability significantly improved with a 69% reduction in motion command jitter (from 74 times to 23 times), resulting in 62% faster mission completion (from 15.17 seconds to 5.76 seconds).

The proposed method offers a cost-effective, software-based solution that can be readily adopted by other KRSBI teams facing similar hardware limitations. Unlike previous works [3], our method addresses both computational efficiency and tracking continuity simultaneously.

Despite the significant improvements achieved, it is important to acknowledge the limitations of this study. First, the system was evaluated exclusively on a single laptop model (ASUS K401U) and one specific omnidirectional camera setup; therefore, processing performance and frame rates may vary on different hardware configurations. Second, the implemented Kalman Filter relies on a linear constant-velocity motion model, which cannot anticipate sudden non-linear changes in the ball's direction, such as bouncing or colliding with other robots or obstacles. Third, the current filter design does not utilize the confidence score from YOLOv8 to adaptively weigh the measurements during the correction phase.

Finally, the experiments were conducted in a controlled laboratory environment, which may not fully replicate the unpredictable dynamics and extreme visual clutter of an actual multi-robot competition.

For future work, several recommendations can be proposed to further enhance the system's capabilities. First, testing the architecture across a wider range of consumer laptops and embedded edge platforms, such as the NVIDIA Jetson [18] or Raspberry Pi equipped with a Neural Processing Unit (NPU) [19], is recommended to assess its generalizability. Second, implementing an Extended Kalman Filter (EKF) [20] could provide better handling of the ball's non-linear dynamics, such as sudden bounces. Third, incorporating the YOLOv8 detection confidence score directly into the Kalman gain computation, similar to the ConfTrack approach [21], would help the system adaptively reject low-quality measurements. Finally, extending the framework to support multi-object tracking (including the ball, goals, and opponent robots) and validating the system in official, multi-robot KRSBI matches will be crucial for its deployment in real-world competitions.

ACKNOWLEDGEMENT

The authors would like to express their sincere gratitude to all parties who have supported the completion of this research and the writing of this article. Special thanks are extended to the robotics team members who assisted during the data collection and experimentation phases. Their contributions, encouragement, and assistance are deeply appreciated.

REFERENCES

- [1] B. Kusumoputro *et al.*, “Pedoman Kontes Robot Indonesia (Kri) Pendidikan Tinggi Tahun 2024,” 2024.
- [2] B. M. Nanda, S. Siregar, and M. I. Sani, “Implementasi Object Detection Pada Robot Sepak Bola Beroda Berbasis Kamera Omnidirectional Menggunakan OpenCV,” vol. 9, no. 4, pp. 2064–2068, 2023.
- [3] T. M. Farhan and F. Candra, “CNN-Based Ball and Goal Detection for KRSBI Robot with Omnidirectional Camera,” *Int. J. Electr. Energy Power Syst. Eng.*, vol. 8, no. 2, pp. 86–98, 2025, doi: 10.31258/ijeepse.8.2.1-13.
- [4] M. Surya, A. Nehemia Toscani, C. Saputra, Y. Pratama, and M. I. Bustami, “Penggunaan Yolo Untuk Deteksi Robot Dan Gawang Pada Robot Sepak Bola Beroda,” *Indones. J. Comput. Sci.*, vol. 14, no. 1, pp. 1055–1070, 2025, doi: 10.33022/ijcs.v14i1.4575.
- [5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-time Object Detection,” *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, vol. 2016-Decem, pp. 779–788, 2016, doi: 10.1109/CVPR.2016.91.
- [6] M. Yaseen, “What Is Yolov8: An In-Depth Exploration Of The Internal Features Of The Next-Generation Object Detector,” vol. 8, pp. 1–10, 2024.
- [7] Ultralytics, “Kalman Filter (KF).” [Online]. Available: <https://www.ultralytics.com/glossary/kalman-filter-kf>
- [8] S. E. Ryu and K. Y. Chung, “Detection Model Of Occluded Object Based On Yolo Using Hard-example Mining And Augmentation Policy Optimization,” *Appl. Sci.*, vol. 11, no. 15, 2021, doi: 10.3390/app11157093.
- [9] J. Nuralim, N. Fath, A. Musafa, Sujono, and D. S. Broto, “Perancangan Sistem Pendeteksian Obyek Bola Dengan Metode Framework YOLOv4,” *J. Maest.*, vol. 5, no. 2, pp. 289–294, 2022, [Online]. Available: <https://jom.ft.budiluhur.ac.id/index.php/maestro/article/view/531>
- [10] C. Saputra, “Implementasi Algoritma SIFT (Scale-Invariant Feature Transform) Dan Algoritma Kalman Filter Dalam Mendeteksi Objek Bola,” *J. Process.*, vol. 18, no. 1, pp. 73–82, 2023, doi: 10.33998/processor.2023.18.1.791.
- [11] R. Sholehurrohman, M. R. Habibi, I. S. Ilman, R. Taufiq, and M. Muhaqiqin, “Analisis Metode Kalman Filter, Particle Filter dan Correlation Filter Untuk Pelacakan Objek,” *Komputika J. Sist. Komput.*, vol. 12, no. 2, pp. 21–28, 2023, doi: 10.34010/komputika.v12i2.9567.
- [12] L. E. Taylor, M. Mirdanies, and R. P. Saputra, “Optimized Object Tracking Technique Using Kalman,” vol. 07, pp. 57–66, 2016, doi: 10.14203/j.mev.2016.v7.57-66.

- [13] Y. G. Lee, "Confidence-Guided Frame Skipping to Enhance Object Tracking Speed," *Sensors*, vol. 24, no. 24, 2024, doi: 10.3390/s24248120.
- [14] M. Quigley *et al.*, "ROS : An Open-source Robot Operating System," no. Figure 1, 2009.
- [15] OpenCV, "Open Source Computer Vision Library." [Online]. Available: <https://opencv.org/>
- [16] NumPy, "NumPy." [Online]. Available: <https://numpy.org/>
- [17] K. E. I. Takura, Y. U. M. A. N. Arita, and S. H. N. Oaki, "Automatic pear and apple detection by videos using deep learning and a Kalman filter," vol. 4, no. 5, pp. 1688–1695, 2021.
- [18] S. Ramos-sanchez, J. Lezama, R. Yauri, and J. Zevallos, "Object Detection on Road : Vehicle ' s Detection Based on Re-Training Models on NVIDIA-Jetson Platform," 2026.
- [19] Y. Fastiuk, "Performance Evaluation Of Object Detection For Traffic Rule Violation Monitoring On Raspberry Pi With NPU," pp. 137–141, 2026, doi: 10.62731/mcnd-13.02.2026.004.
- [20] A. Moksyakov, Y. Wu, S. A. Gadsden, J. Yawney, and M. AlShabi, "Object Detection and Tracking with YOLO and the Sliding Innovation Filter," *Sensors*, vol. 24, no. 7, pp. 1–18, 2024, doi: 10.3390/s24072107.
- [21] H. Jung, S. Kang, T. Kim, H. Kim, H. L. Klemove, and R. Korea, "ConfTrack : Kalman Filter-based Multi-Person Tracking by Utilizing Confidence Score of Detection Box," *CVF Open Access*, pp. 6583–6592, 2024.